

FINAL YEAR AIML PROJECT

DocuTrust AI

A Citation-Grounded Multimodal RAG System for Reliable Policy and Compliance Intelligence

DocuTrust AI turns organizational documents into a trustworthy, searchable knowledge base. Every answer is linked to source evidence instead of being presented as an unsupported chatbot response.

Project type	Full-stack AIML document intelligence application
Core focus	Grounded retrieval, OCR, citations, analytics, and auditability
Primary users	Organizations, compliance teams, HR, operations, and students

Prepared for project explanation, reviews, placement interviews, and viva preparation.

1. Executive Summary

DocuTrust AI is an AI-powered document intelligence platform. Users upload PDFs, Word documents, text files, or image documents such as bills and receipts. The system extracts text, indexes it as evidence chunks, answers natural-language questions, and displays source citations with page information.

The project is designed to reduce a key risk of generic chatbots: hallucination. Rather than relying only on a language model, DocuTrust retrieves relevant evidence first and returns a grounded answer. When evidence is weak, it responds with an insufficient-evidence message instead of guessing.

Core value proposition: trustworthy answers from uploaded documents, supported by evidence that the user can verify.

2. Problem Statement and Objectives

Organizations store important information across policies, SOPs, invoices, contracts, compliance manuals, and circulars. Manual search is slow, while general chatbots may answer incorrectly without proof.

Problem statement: How can an AI assistant answer questions from organizational documents while preserving traceability, showing evidence, and refusing unsupported answers?

Project objectives

- Accept common enterprise document formats and extract searchable text.
- Use OCR for image-based bills, receipts, and scans.
- Retrieve the most relevant evidence for a natural-language question.
- Return concise answers with document and page citations.
- Use confidence and abstention to reduce unsupported answers.
- Provide audit, analytics, document management, and a usable web interface.

3. Architecture and End-to-End Workflow

Stage	What happens
1. Upload	User uploads PDF, DOCX, TXT, Markdown, or image file.
2. Extract	Native parsers extract PDF/DOCX text; Tesseract OCR reads image text.
3. Chunk	Text is divided into smaller chunks with document and page metadata.
4. Index	Document metadata and chunks are stored in SQLite.
5. Retrieve	Question words are compared with chunks and ranked by relevance.
6. Decide	Confidence determines grounded answer vs. insufficient evidence.
7. Respond	System returns concise answer, evidence, page citation, and audit event.

Architecture summary

Upload -> Extract/OCR -> Chunk + metadata -> SQLite -> Retrieve -> Confidence gate -> Grounded answer + citation -> Audit and analytics

4. Core AIML Approach

DocuTrust follows a Retrieval-Augmented Generation (RAG) style workflow. RAG has two stages: retrieval finds relevant source content, and generation creates an answer from that retrieved content.

Mode	How it works	Why it matters
Offline grounded mode	Creates a structured answer directly from retrieved source text.	Works without API cost and does not invent unsupported prose.
Optional LLM mode	Sends only retrieved evidence to an OpenAI-compatible model for synthesis.	Produces more natural language while keeping the answer source constrained.

The current retrieval baseline is intentionally explainable. It uses tokenization, stop-word removal, basic word normalization, and a TF-IDF-inspired relevance score. This is a strong baseline for a final-year project because it can be measured and later compared against embeddings.

Important technical honesty: the current version is not yet a vector-database semantic search system. Adding Sentence Transformers, embeddings, FAISS/ChromaDB, and reranking is an important planned enhancement.

5. Document Processing and OCR

Input	Library or Engine	Result
PDF	PyPDF	Native page-level text extraction
DOCX	python-docx	Paragraph-level extraction
TXT / Markdown	Python text decoding	Direct searchable text
PNG / JPG / TIFF / BMP	Pillow + pytesseract + Tesseract	OCR text extraction

After extraction, the system divides text into chunks. Each chunk retains chunk ID, document ID, page number, optional section, and text. Chunking enables faster retrieval, smaller LLM context, and page-level citations.

For OCR bills, raw output is often a long unstructured line. DocuTrust detects common invoice fields such as invoice number, billing period, payment due date, and total amount, then formats them as aligned label-value output.

6. Features Implemented

Feature	What it does	Why it is useful
Citation-grounded answers	Shows document name, page number, and focused evidence.	Lets users verify answers and improves trust.
Confidence + abstention	Returns grounded or insufficient-evidence status.	Reduces hallucination risk.
Structured bill extraction	Formats invoice number, period, due date, and amount.	Turns messy OCR into readable information.
Document-scoped queries	Search all sources or only one selected document.	Avoids mixing unrelated document content.
Source document management	Delete one document or clear all sources with confirmation.	Supports removal of old or irrelevant documents.
Audit trail	Stores question, status, confidence, timestamp, and citation IDs.	Provides traceability and accountability.
Audit entry deletion	Delete a single audit record using a dustbin action.	Lets users manage query-history data.
Workspace analytics	Shows documents, chunks, grounded-answer rate, and average confidence.	Measures usage and answer quality.
Light / dark theme	Theme selector is saved in browser local storage.	Improves usability and product polish.
Swagger API documentation	FastAPI exposes interactive documentation at /docs.	Simplifies testing and demonstrates API engineering.

7. Database and Audit Design

DocuTrust uses SQLite because it is lightweight, local, easy to demonstrate, and appropriate for an MVP. A future production deployment can use PostgreSQL.

Table	Important fields	Purpose
documents	id, filename, media type, page count, extraction method	Stores source-document metadata.
chunks	document ID, page number, section, text	Stores searchable evidence chunks.
query_events	question, status, confidence, citation IDs, time	Stores audit trail and analytics data.

The analytics dashboard uses database aggregates to calculate document count, evidence-chunk count, query count, grounded-answer rate, and average confidence. This changes the project from a simple demo into a measurable system.

8. Backend APIs

Endpoint	Purpose
GET /	Open the user-facing DocuTrust application.
GET /health	Check service status, LLM configuration, and document count.
POST /api/v1/documents	Upload, extract, chunk, and index a document.
GET /api/v1/documents	List indexed source documents.
DELETE /api/v1/documents/{id}	Delete one source document and its chunks.
DELETE /api/v1/documents	Delete all source documents.
POST /api/v1/query	Run document-scoped retrieval and return answer/citations.
GET /api/v1/analytics	Return workspace quality metrics.
GET /api/v1/query-history	Return recent audit events.
DELETE /api/v1/query-history/{id}	Delete a selected audit entry.
GET /docs	Open Swagger interactive API documentation.

9. Frontend and User Experience

The frontend is implemented with HTML, CSS, and vanilla JavaScript. It communicates with FastAPI using fetch requests. Keeping the frontend dependency-light makes local execution and viva explanation easier.

- Upload panel and source-document list.
- Dustbin actions for document and audit deletion, always protected by confirmation.
- Document scope selector for precise retrieval.
- Aligned answer cards and source evidence blocks.
- Analytics dashboard and reusable recent questions.

- Responsive layout with persistent light/dark theme selection.

10. Security and Reliability

Control	Implementation
API key isolation	OPENAI_API_KEY is stored in local .env and never exposed to browser JavaScript.
Git safety	.env and local data are excluded using .gitignore.
Upload protection	Filenames are sanitized and upload size is limited.
Deletion safety	Document and audit deletion requires browser confirmation.
Answer safety	Weak retrieval causes insufficient-evidence response rather than a guess.
LLM resilience	Optional LLM failures fall back to evidence-first local answers.

11. Testing and Validation

Core project checks cover relevant retrieval, unsupported-question abstention, numeric confidence handling, tax validity extraction, structured bill extraction, document deletion, audit deletion, analytics calculations, and API-level query behaviour.

The interface was also reviewed after implementing aligned bill fields, citations, dark mode, analytics, source deletion, and audit deletion.

12. Limitations and Future Scope

Current limitation	Recommended next upgrade
Lexical retrieval baseline	Add sentence embeddings, FAISS/ChromaDB, and hybrid search.
No user authentication	Add JWT login, role-based access control, and document ACLs.
No PDF source viewer	Add a PDF viewer with highlighted evidence passage.
Basic OCR structure	Add layout-aware OCR, table extraction, and scanned-PDF OCR.
SQLite MVP database	Use PostgreSQL for production multi-user deployment.
Prompt-based grounding in LLM mode	Add automated claim-to-citation verification.

13. Evaluation Plan for Report or Paper

Create a benchmark of approximately 10-20 public or synthetic policy documents and 75-100 labelled questions. Record the expected source page for each question.

Metric	Definition	Why it matters
Retrieval Recall@5	Correct source appears in top 5 retrieved chunks.	Measures retrieval quality.
Citation precision	Cited passage truly supports answer.	Measures trustworthiness.

Metric	Definition	Why it matters
Faithfulness	Answer claims are supported by evidence.	Measures hallucination reduction.
Abstention accuracy	Unsupported questions are correctly refused.	Measures safety behaviour.
Latency	Time required to return an answer.	Measures user experience.

14. Placement and Viva Explanation

I developed DocuTrust AI, a citation-grounded document intelligence platform. It extracts text from PDFs, Word files, text files, and images; indexes the content as page-aware chunks; retrieves relevant evidence for a natural-language question; and returns a cited answer with confidence. It also supports OCR bill formatting, document-scoped search, audit history, analytics, document lifecycle management, and optional LLM synthesis constrained to retrieved evidence.

Why it is placement-level

- Combines AIML concepts with backend engineering and product UX.
- Solves a real enterprise problem rather than only demonstrating a generic chatbot.
- Includes reliability controls: citations, confidence, abstention, audit trail, and analytics.
- Has measurable future experimentation paths: embeddings, vector search, reranking, and evaluation metrics.
- Demonstrates API design, database design, OCR, deployment readiness, and secure secret handling.

Common viva questions

Question	Short answer
What is RAG?	Retrieval-Augmented Generation retrieves source evidence before generating an answer.
Why citations?	They let users verify exactly where the answer came from.
Why abstain?	A safe system should not invent an answer when evidence is missing.
Why SQLite?	It is lightweight and suitable for a local MVP; PostgreSQL is planned for production.
How is the API key protected?	It is stored only in backend .env configuration and is never sent to the browser.
What is the next ML upgrade?	Embedding-based hybrid retrieval with a vector database and reranker.

End of report